

Veinte Artículos

Keiner Andrés Cano Narváez

Instructor: Jesús Ariel

Artículo 1: Patrones de diseño

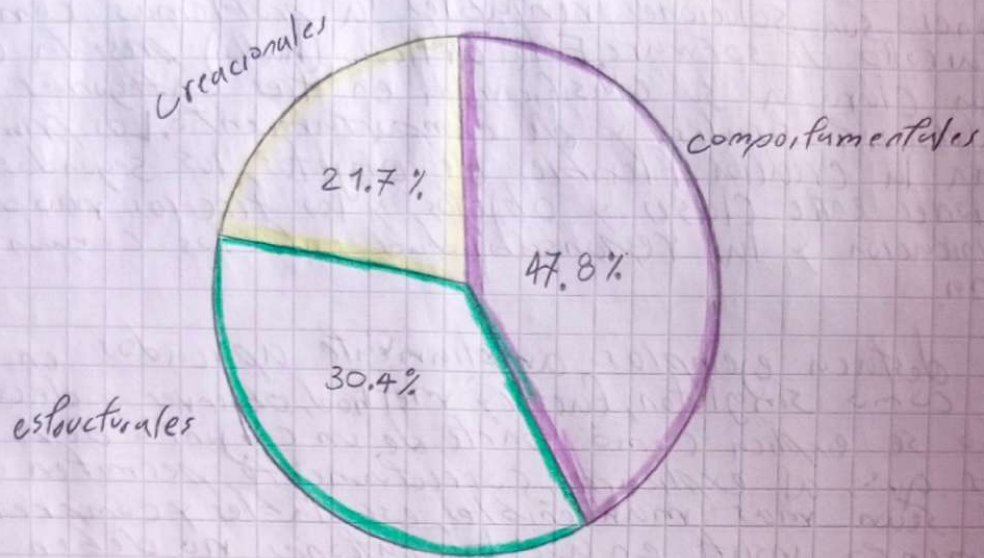
Los patrones son soluciones reutilizables a problemas comunes en el desarrollo de software. Este artículo (hdtla) presenta una introducción clara a su clasificación en tres categorías principales: Creacionales, estructurales y de comportamiento. Los primeros simplifican la creación flexible de objetos, los segundos organizan las relaciones entre clases y objetos, y los terceros modelan la comunicación y la responsabilidad entre los componentes del sistema.

El texto destaca ejemplos ampliamente aplicados en la práctica, como singleton, factory method, observer y decorator. Cada uno se explica como parte de un conjunto de buenas prácticas que, al explicarse correctamente, permiten que los sistemas sean más mantenibles, escalables y comprensibles. El documento insiste en que los patrones no deben verse como recetas estrictas, sino como guías adaptables a cada proyecto, advirtiendo (cu) que un uso excesivo puede generar complejidad innecesaria.

Además subraya su relevancia en la formación académica de los ingenieros de software, ya que ofrecen un lenguaje común para describir soluciones de diseño, facilitando la comunicación entre equipos y la transferencia de conocimiento.

En conclusión, los patrones son un recurso esencial para mejorar la calidad de software, ya que ofrecen un lenguaje común para describir el diseño de soluciones. Su aplicación (favoc) favorece la adaptabilidad en entornos tecnológicos en constante evolución, convirtiéndose en una herramienta duradera y sólida.

Distribución de patrones de diseño



La (Cruz) clasificación de los patrones de diseño muestra que la mayoría son de tipo comportamentales, lo que refleja la importancia de definir de manera clara como los objetos colaboran y se comunican dentro de un sistema. En el desarrollo de software, la interacción entre componentes suele ser más compleja que su simple creación o estructuración, de ahí la abundancia de este tipo de patrones. Por otra parte, los estructurales desempeñan un papel esencial al permitir arquitecturas más organizadas, flexibles y extensibles, facilitando la creación y reutilización de código. Aunque los creacionales son menos en número, su influencia es significativa, ya que determinan la manera en como se instancian los objetos. En conjunto la clasificación evidencia la importancia de equilibrar creación, estructura y comunicación.

Artículo 2: Introducción a la programación Orientada a objetos

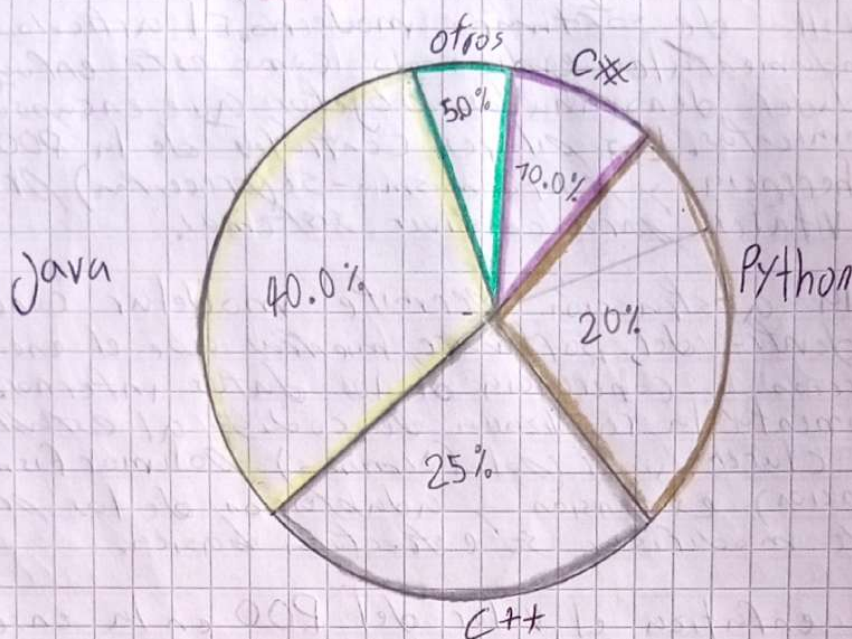
Constituye uno de los paradigmas más influyentes en el desarrollo de software moderno. El artículo introduce sus principios fundamentales, explicando cómo este enfoque organiza en torno a entidades denominadas objetos, que encapsulan datos y comportamientos. Los pilares centrales de la POO - abstracción, encapsulamiento, herencia y poliformismo - se presentan como elementos vitales para construir sistemas.

Se resalta que la abstracción permite modelar conceptos del mundo real dentro del software, mientras que el encapsulamiento asegura la protección y coherencia de los datos internos. La herencia, por su parte, fomenta la reutilización de código al establecer jerarquías entre clases, y el (poliformismo) polimorfismo facilita la extensión y adaptación de los programas sin necesidad de modificar su estructura básica.

El texto también enfatiza el valor del POO en la enseñanza de la programación, pues brinda a los estudiantes una manera intuitiva de razonar sobre problemas complejos.

En conclusión, el artículo posiciona a la POO como un paradigma esencial para la construcción de software de calidad, capaz de responder a los retos de escalabilidad y mantenibilidad propios de las aplicaciones actuales. Gracias a sus principios, los desarrolladores logran producir sistemas más modulares, adaptables y alineados con la lógica de los problemas que buscan resolver.

Lenguajes más usados en la enseñanza de POO



La (Po) programación orientada a objetos representa un paradigma transformador que trasciende la mera escritura de código. Mediante el uso de Clases como plantillas y objetos como instancias concretas, permite organizar el software de manera más natural y modular. El encapsulamiento garantiza el estado interno se protege y solo se interactúa mediante interfaces controladas. La herencia permite la reutilización eficiente de comportamiento, mientras que el polimorfismo añade flexibilidad al permitir que entidades heterogéneas compartan comportamientos comunes. La abstracción elimina enormes detalles de implementación, favoreciendo el enfoque esencial del sistema. En conjunto, estos principios fomentan un diseño más flexible, mantenible y escalable, facilitando el manejo de la complejidad inherente al desarrollo del software moderno.

Artículo 3: Estudio comparativo de Flutter y React Native

Este estudio analiza y compara dos frameworks multiplataforma para el desarrollo de aplicaciones móviles, Flutter y React Native, con el propósito de revelar sus diferencias en rendimiento, facilidad de uso, capacidades y ciclo de vida de desarrollo.

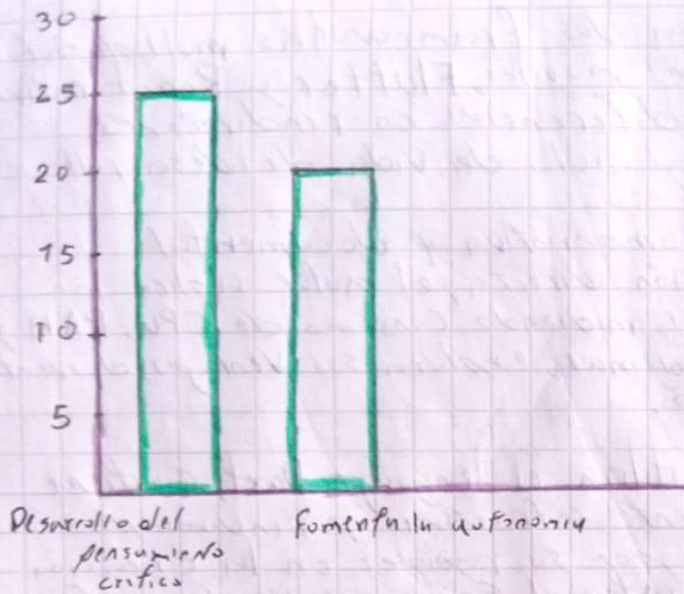
Basado en una (f) metodología comparativa y documental complementada con observación directa, el autor aplica los frameworks a ejemplos prácticos, midiendo consumo de CPU, RAM y tiempos de (comp) compilación. Además, explora sus ventajas, desventajas, sinérgico y procesos de instalación.

Flutter, fue creado por Google, utiliza el lenguaje Dart, contiene widgets para interfaces visualmente atractivas y rindimiento cercano al nativo. Se destaca por su rapidez en la creación de proyectos y su documentación integral. Entre sus ventajas figuran la base de código única por varias plataformas, simplicidad en el aprendizaje y velocidad de desarrollo. Como desventaja para desarrolladores con experiencia en JavaScript se menciona una relativa novedad y comunidad aún pequeña.

Por otro lado, React Native, de Facebook, emplea JavaScript y permite compartir código entre plataformas. Ofrece ventajas como (compatibilidad) compatibilidad multiplataforma, hot (reload) reload, uso de APIs nativas sin (webw) WebView, y menor curva de aprendizaje para desarrolladores con experiencia en JavaScript. Sus desventajas incluyen dependencia de librerías externas, mensajes de error poco claros y ser aún beta en algunos componentes.

La comparación final incluye tablas sobre rendimiento, herramientas, curva de aprendizaje, documentación y tiempos de ejecución. En general, el documento concluye con que Flutter sobresale en rendimiento y soporte técnico, mientras que React Native resulta accesible para quienes ya dominan JavaScript. La decisión entre ambos debe basarse en el contexto o experiencia del equipo.

Percentage %



En el ámbito del desarrollo móvil (cross platform), frameworks como Flutter y React Native representan una evolución significativa frente a los métodos tradicionales. La elección entre ellos no solo depende de la popularidad, sino también de las necesidades técnicas y del contexto de cada proyecto. Flutter, con su crecimiento en adopción, ofrece una interfaz más unificada y rendimiento cercano al nativo, lo que lo convierte en una alternativa sólida para (cross) aplicaciones que buscan escalabilidad y consistencia visual. Por otro lado, React Native mantiene relevancia gracias a su integración con el ecosistema JavaScript lo que facilita la adopción en equipos con experiencia web. En conjunto, estas herramientas muestran como la innovación tecnológica responde a la demanda de soluciones más eficientes y accesibles en el desarrollo actual.

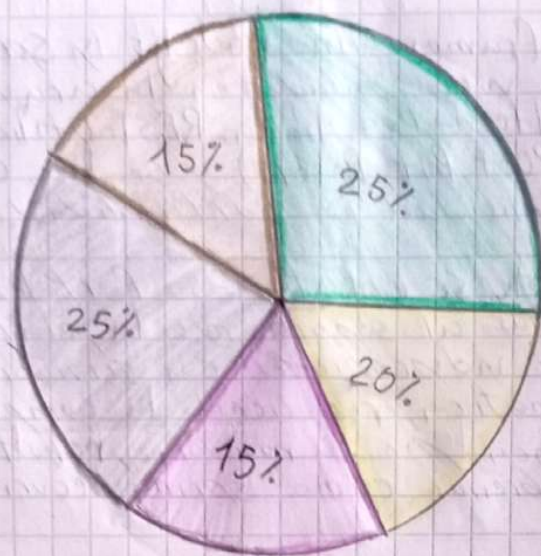
Artículo 4: Node.js vs Spring Boot en el desarrollo Backend de aplicaciones Web

El artículo compara los frameworks Node.js y Spring Boot para el desarrollo de servicios web basados en la arquitectura REST, presentada como una alternativa ligera. REST utiliza métodos explícitos de HTTP (Get, Post, Put, Delete) y se ha convertido en un estándar por su simplicidad y portabilidad.

En el caso de Spring Boot, se enfatiza su portabilidad en el entorno empresarial, (gracias) gracias al soporte de la plataforma Java. Permite crear aplicaciones independientes con servidores embebidos y (con) configuración automática, reduciendo la complejidad del despliegue. Además se destaca por su madurez, robustez y seguridad, y que cuenta con librerías como Spring Security y compatibilidad.

Por otro lado, Node.js aprovecha el modelo de ejecución asíncrono de JavaScript (ocurre en tiempo real) y combinado con Express.js, ofrece rapidez y escalabilidad. Su facilidad de aprendizaje, junto con la integración con base de datos NoSQL como MongoDB, lo hacen atractivo para proyectos que requieren flexibilidad y velocidad. No obstante, presenta limitaciones en el área de (segunda) seguridad, debido a la menor variedad de librerías especializadas y depende más de paquetes (pequeños) externos.

El análisis concluye que ambos frameworks permite crear servicios REST en poco tiempo. Node.js es más conveniente para proyectos ágiles y escalables, mientras Spring Boot resulta más adecuado para aplicaciones empresariales que exigen seguridad, estabilidad e integración sólida con base de datos relacionales.



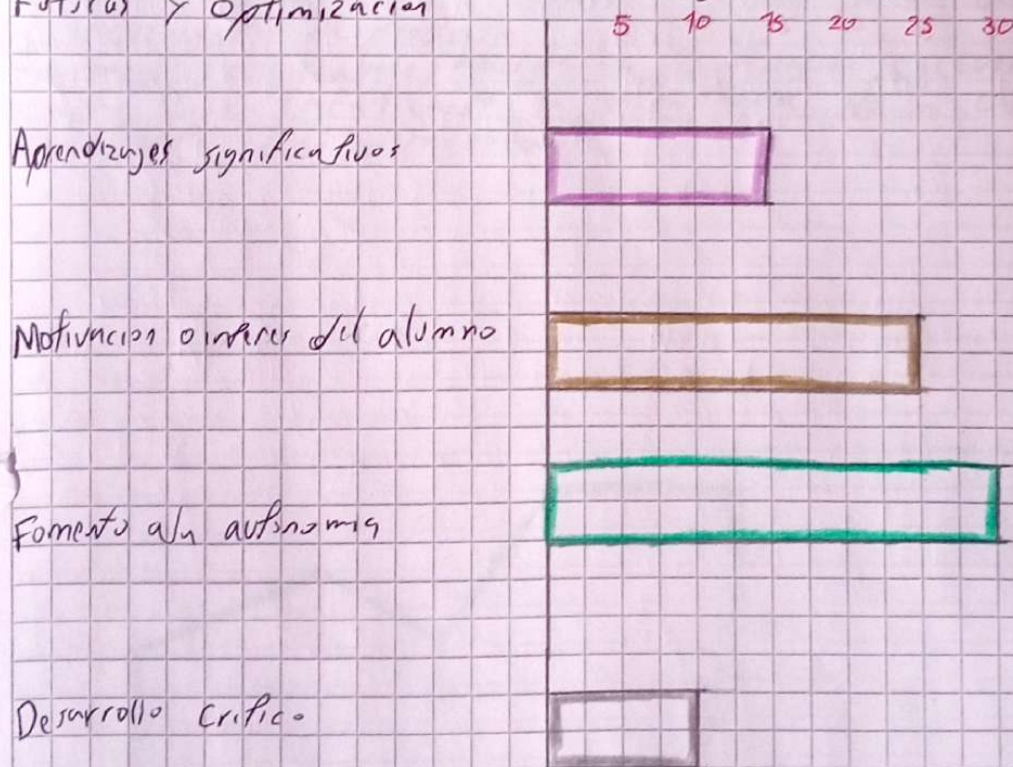
Desarrollo del pensamiento crítico ● Fomenta la autonomía
 inclusión y atención a la diversidad ● Motivación e interés del alumno ●
 Aprendizajes significativos ●

La implementación de servicios web Restful sobre un backend con Spring boot evidencian una evidencia clara en el desarrollo moderno de aplicaciones: la transición hacia arquitecturas modulares (exuberantes) escalables y descentralizadas. Spring Boot facilita la creación de microservicios sólidos al proporcionar una configuración rápida, componentes integrados y un ecosistema maduro. Los servicios Restful, por su parte, permiten una comunicación clara y sencilla desacoplada entre sistemas, favoreciendo la interoperabilidad y facilitando el mantenimiento. Este enfoque potencia la agilidad del desarrollo, al mismo tiempo que mejora la capacidad de adaptarse a nuevos requisitos o cargas de tráfico.

(Artículo) Artículo 5: Diseño e implementación de un microservicio con Spring

Aborda el proceso completo de desarrollo de un microservicio, desde el diseño de arquitectura hasta la implementación y pruebas. El documento analiza diferentes enfoques y concluye que la arquitectura de microservicios ofrece mayor flexibilidad y escalabilidad frente a soluciones monolíticas. Para la implementación, se utiliza Spring Boot como framework principal, junto con Spring Data para la persistencia de datos y MongoDB/CosmosDB como base de datos NoSQL. Además, se integra Swagger para documentar y probar la (AB) API de forma eficiente.

El microservicio desarrollado gestiona información de lista de reproducción y usuario Spotify, permitiendo filtrar aquellas listas de canciones escuchadas. Usa Maven y Lombok para optimizar el desarrollo, organiza la aplicación en capas y aplica DTOs y Java Stream. Además, analiza costes, impacto socioeconómico, regulaciones y plantea mejoras futuras y optimización.



El desarrollo de aplicaciones (web) backend mediante servicios web RESTful continúa hoy en día un estado en la industria del software, gracias a su capacidad para ofrecer soluciones ligeras, flexibles y fáciles de integrar. El modelo RESTful al basarse en principios como el uso claro de métodos HTTP, la identificación de recursos mediante URIs, la independencia de los estados, permite construir APIs consistentes y de amplia interoperabilidad. Además frameworks modernos como Springboot han simplificado su implementación, brindando a los desarrolladores herramientas que aceleran los procesos y aseguran la calidad de los productos. Esta evolución refleja como la (tecnología) tecnológica responde a la necesidad de aplicaciones ágiles, adaptables y sostenibles, marcando un camino hacia sistemas más eficientes y orientados a la colaboración digital.

Artículo 6: Metodología Scrum

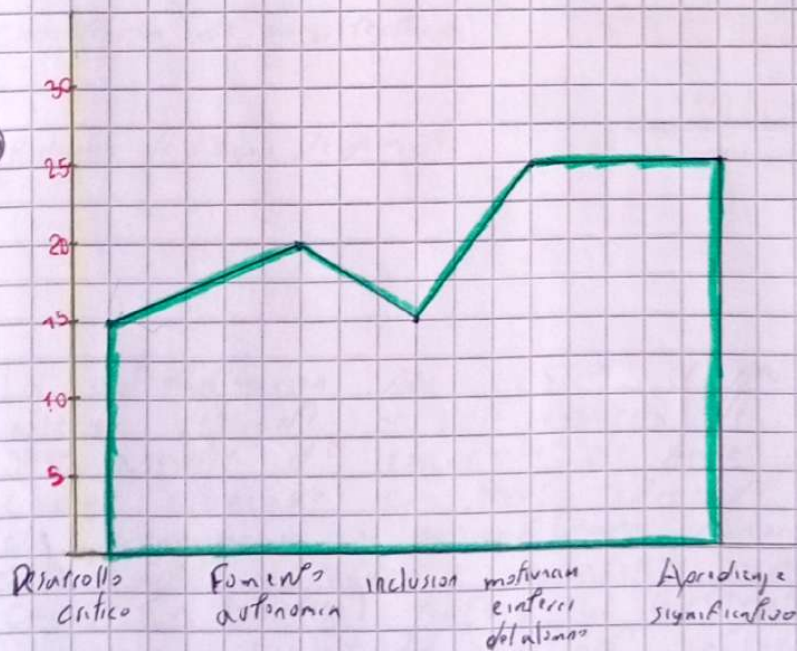
El documento presenta un análisis profundo sobre la gestión de proyectos informáticos mediante el uso de metodologías ágiles, centrándose especialmente en Scrum. Inicia explicando los fundamentos de gestión de proyectos y limitaciones de los métodos tradicionales como la metodología en cascada, resultando poco flexible frente a entornos cambiantes.

Posteriormente introduce las metodologías ágiles y su enfoque iterativo, colaborativo y adaptable, destacando sus ventajas: entregas frecuentes, comunicación constante con el cliente, reducción de riesgos y mayor capacidad de respuesta ante cambios. Entre estas metodologías, Scrum se describe como un marco de trabajo que organiza el desarrollo en sprints.

El documento detalla los roles principales: Product Owner, encargado de priorizar los requisitos; Scrum Master, facilita el proceso; y equipo de desarrollo, responsable de implementar las funcionalidades.

También se explica los elementos clave: Product Backlog, Sprint Backlog e incrementos de producto.

Finalmente, el trabajo aborda la planificación, ejecución y revisión de cada fase enfatizando la importancia de la colaboración entre el cliente y el desarrollador.

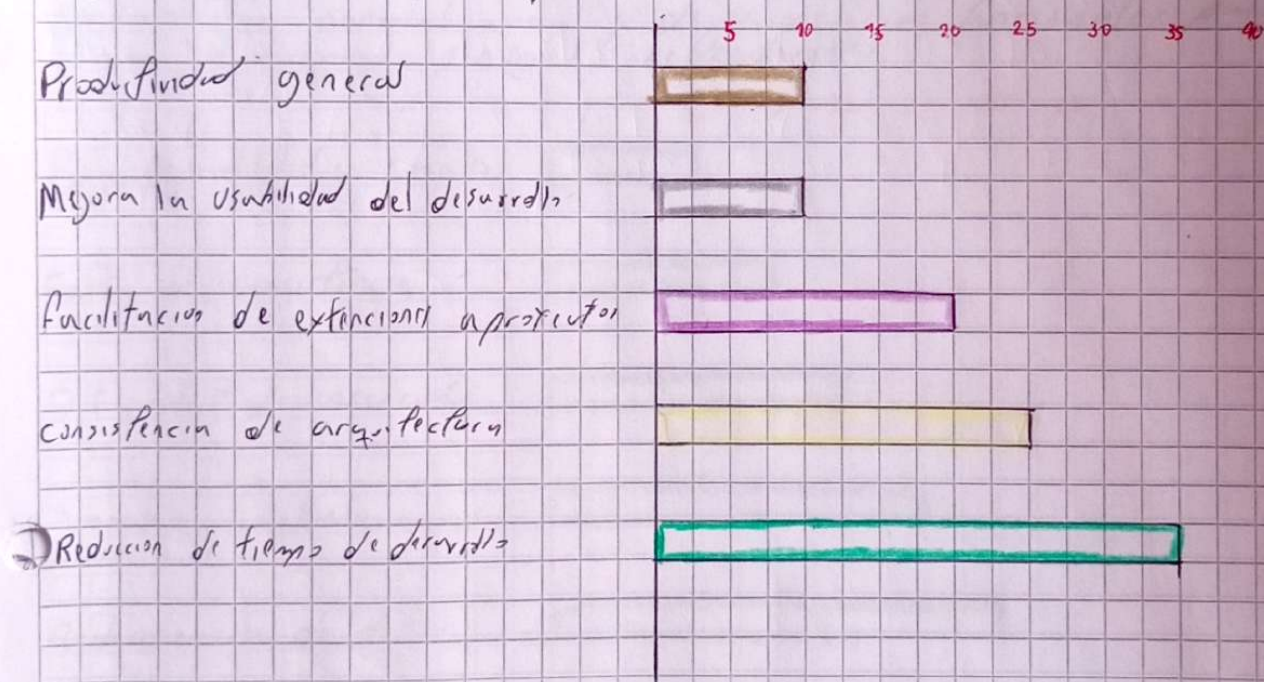


El trabajo analiza la construcción de un sistema de comunicación para una (empírica) empresa de transporte urbano orientado a mejorar la eficiencia y la coordinación interna. La propuesta integra tecnologías de información para optimizar la gestión de recursos, reducir tiempos de respuesta y facilitar la toma de decisiones en entornos cambiantes. Este enfoque refleja la importancia de adoptar la infraestructura tecnológica a las necesidades concretas de cada organización, garantizando tanto la operatividad diaria como la escalabilidad futura.

7 Automatización de la codificación (MVC)

El artículo presenta Gabyo MVC, una herramienta en Java que automatiza la codificación de patrón modelo-vista-controlador (MVC) partiendo de esquemas de bases de datos en MySQL. Usa scripts generados con DB Designer o MySQL que primero homogeniza; luego un analizador sintáctico identifica tablas, claves y vistas, construyendo una estructura dinámica que modela relaciones y dependencias. Con reglas configurables en archivos XML el sistema genera automáticamente clases DTO, así como capas. Habilitando proyectos nuevos o subproyectos acoplables a sistemas existentes sin reestructuras.

La interfaz permite seleccionar tablas y atributos tras lo cual produce paquetes Java documentados y listos para entregar con frameworks de presentación. Las pruebas muestran reducciones a tiempo del 66%-70% en un caso de 57 tablas, la generación toma segundos mejorando consistencia y mantenibilidad.



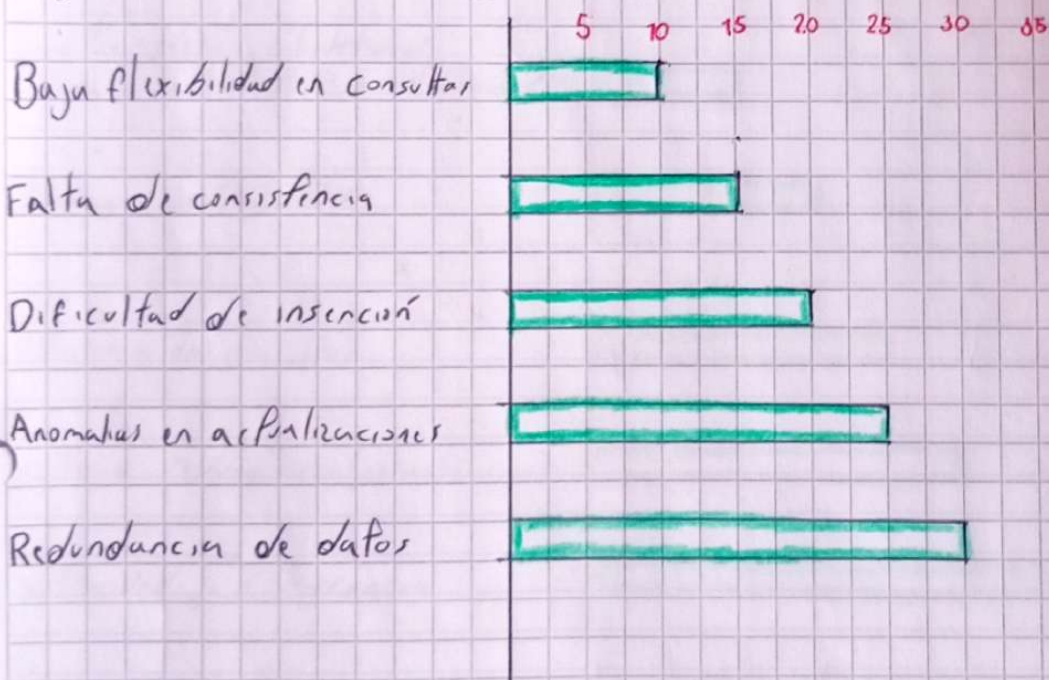
La automatización para generar el patrón MVC supone un avance relevante en la ingeniería de software. Con la herramienta Java, a partir de esquemas de base de datos se genera código alineado con MVC, reduciendo hasta el 70% el tiempo de programación y garantizando consistencia en proyectos web. Esto facilita extender y mantener aplicaciones sin reescribir código. En síntesis, la automatización mejora la productividad, homogeneidad y escalabilidad, mostrando impactos de patrones de diseño aporta beneficios concretos y sostenibles a lo largo del ciclo de vida del software.

8 fundamentos sobre la gestión de datos

Presenta una visión integral de los sistemas de base de datos, diseñando tanto para cursos introductorios como para quienes desean profundizar. Comienza con una introducción general sobre la evolución, propósito y arquitectura de los sistemas de gestión de bases de datos, destacando su superioridad frente a los sistemas de archivos tradicionales en términos de seguridad, integridad y abstracción.

Explora los modelos fundamentales: el modelo entidad relación, utilizado para modelar conceptos de alto nivel y el modelo relacional, que aborda operaciones mediante álgebra y cálculo relacional.

Luego, cubre aspectos clave del lenguaje SQL, otros lenguajes como QBE y Datalog y la teoría del diseño relacional basada en normalización y dependencias funcionales para lograr esquemas sin (o con) redundancias.



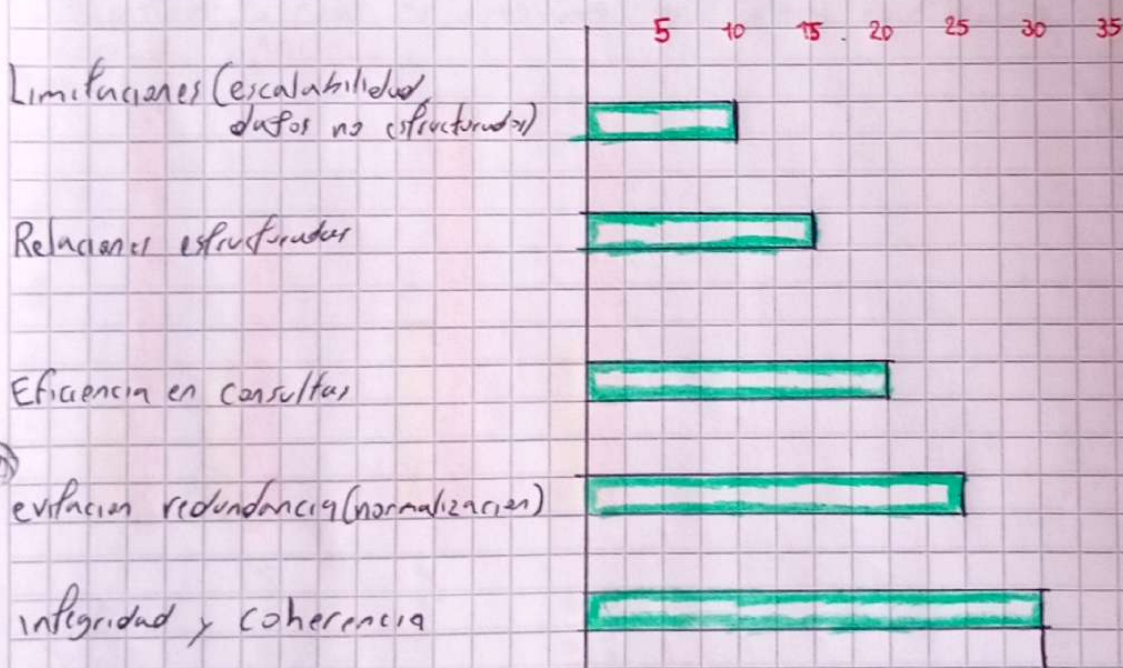
La gestión de base de datos es clave para cualquier sistema de información. Modelos de datos bien definidos y la normalización garantizan consistencia e integridad. Operaciones como consultas, inserciones y actualizaciones, ejecutadas por SGBD, aseguran precisión y disponibilidad. Además, el control de acceso y restricciones (llaves primarias y foráneas) protegen frente a inconsistencias. En conjunto, estos principios permiten construir sistemas robustos, confiables y fáciles de mantener, preparados para responder a demandas de desempeño y seguridad.

9 Principios sobre Base de Datos relacionales

En este documento abordo los fundamentos del modelo relacional de base de datos, desarrollado por Edgar F. Codd a finales de los 60 y se ha convertido en el paradigma más común en la gestión de datos. Se definen como objetivo clave del (manejo) modelo la independencia física y lógica, flexible y uniforme en el manejo de datos.

Hablo de un breve recorrido histórico, señalando hitos como la aparición de DBMS comercial Oracle (1979), SQL (1987) y la estandarización de SQL por ANSI y ISO en los años posteriores. Lo ya nombrado de como se construye por: tablas, filas, atributos, la cardinalidad y dominios de atributos.

Además explico el desarrollo de los objetos relacionales como una evolución híbrida entre el modelo relacional y el orientado a objetos, permitiendo capacidades como herencia, trigger, tipos definidos por el usuario etc.



El modelo relacional es un pilar de la informática al organizar la información en tablas con relaciones y restricciones que garantizan integridad. Basados en reglas, posibilidad de subconjuntos claves, formas normales y la transformación de modelos entidad-relación refuerzan un diseño sólido que evita redundancias. La reflexión central es que la calidad de una base de datos depende más del rigor (central) conceptual y metodológico que del software empleado.

Artículo 10: Propuesta de innovación educativa o didáctica en la enseñanza

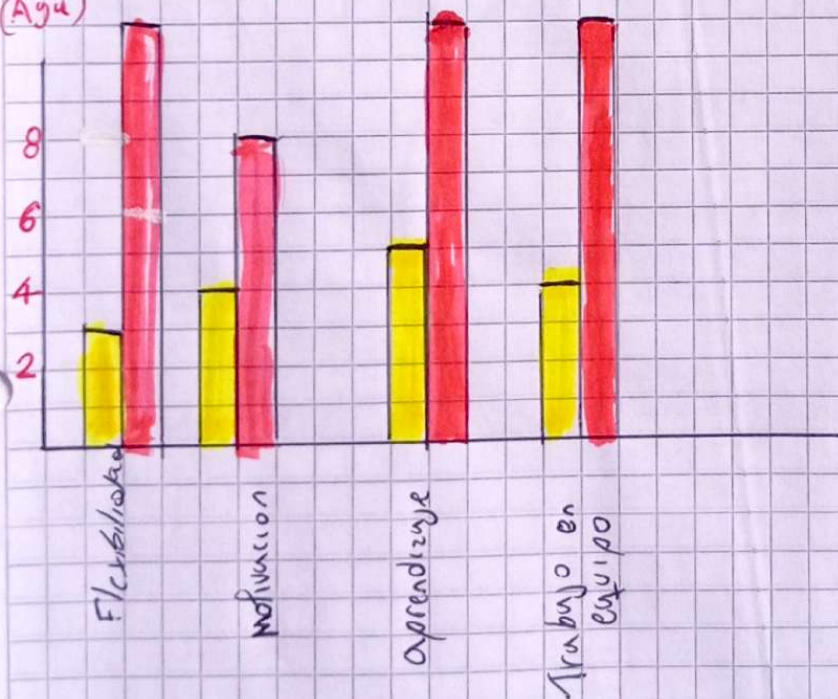
El documento presenta una propuesta de innovar el sistema educativo con procesos de aprendizaje mediante el uso de estrategias didácticas activas y tecnologías educativas. Parte del análisis. El uso de (metodologías) metodologías ágiles facilitan la detección temprana de errores conceptuales y promueven un aprendizaje más práctico y significativo, donde los estudiantes no solo adquieren conocimientos técnicos, sino habilidades blandas como el trabajo en equipo, la comunicación y la gestión del tiempo. El rol del docente cambia de transmisor de información a facilitador del proceso, guiando y apoyando al grupo en cada iteración.

En conclusión, integrar metodologías ágiles en la enseñanza de la programación permite acercar la educación a las dinámicas del mundo laboral, mejorando tanto la comprensión de conceptos como la preparación profesional de los estudiantes. Las metodologías se pueden aplicar en el ámbito educativo y esto promueve la colaboración, la retroalimentación constante y el aprendizaje iterativo, lo cual es bastante útil para estudiantes.

Clásico:

Agil:

(Agu)



La prova replantea el modelo tradicional de enseñanza pasando de un alumno pasivo a uno activo que construye su conocimiento y se prepara para retos reales. Este cambio exige docentes e instituciones dispuestos a formarse y atualizarse, refinar estrategias. La innovación es pedagógica y cultural, no solo tecnológica e implica arriesgar y transformar la mentalidad de los maestros y estudiantes. Adoptar este enfoque puede eleva la calidad educativa y formar ciudadanos críticos, creativos y autónomos.

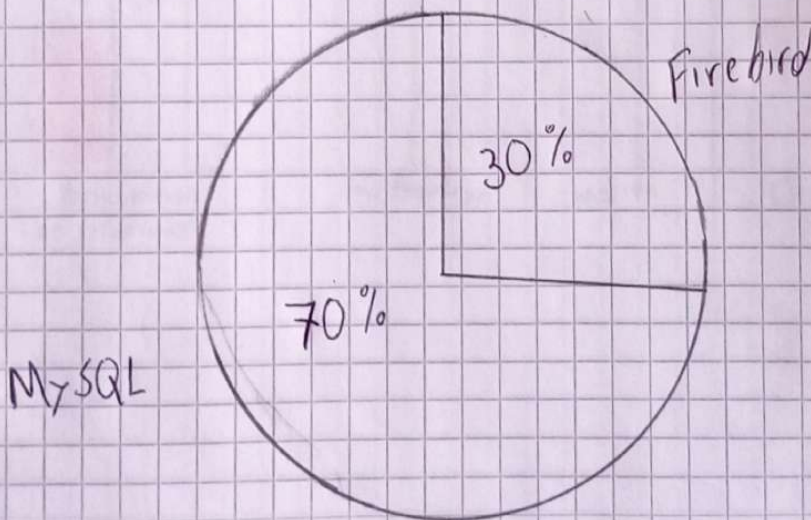


Artículo 17: Evaluación del rendimiento de los motores de bases de datos MySQL y Firebird

Presenta un estudio comparativo entre (a) ambos gestores, utilizando una aplicación genérica de control de inventarios desarrollada en Java y ejecutada sobre Linux. Para realizar las pruebas se diseñó algoritmos encargados de generar cargas masivas de datos respetando la integridad y ajustándose al peso de cada tabla.

Las pruebas se llevaron a cabo tanto en entornos locales como en red, con diferentes configuraciones de (software) hardware, incluyendo procesadores T300 y Pentium 4 y discos duros de diferentes velocidades. Los resultados indicaron que MySQL superó a Firebird en todos los escenarios, mostrando tiempos de respuesta menores en promedio, con una diferencia cercana a un milisegundo. Sin embargo, los autores destacan que más que velocidad, es necesario considerar otras características como la seguridad, procedimientos, almacenamiento y riqueza del lenguaje SQL al seleccionar (una) un motor de base de datos.

Comparación de rendimiento



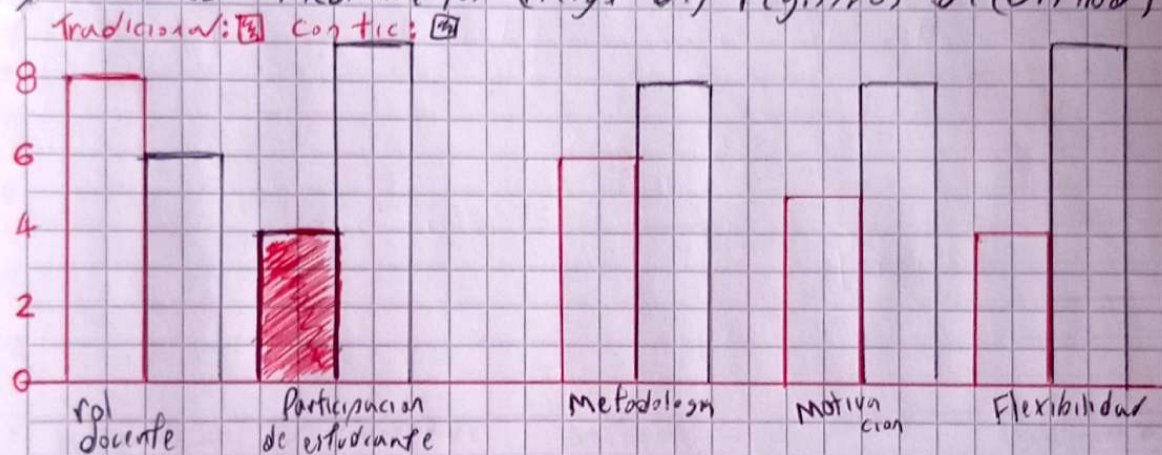
El estudio representa un solo enfoque experimental mediante un programa que mide el rendimiento de MySQL y Firebird en operaciones frecuentes de base de datos. MySQL mostró tiempos más rápidos, aunque los autores subrayan que no existe un ganador absoluto: la elección debe basarse en fortalezas contextuales como seguridad o lenguaje procedural. Además resalta la importancia del software libre y la versatilidad del método para evaluar o mejorar otros sistemas de gestión de datos.

Artículo 12: Seguimiento de proyectos de programación en la educación

Presenta metodología MESEPP, diseñada para mejorar el control y una evaluación de proyectos finales en la asignatura de programación. Esta metodología se apoya en GITHUB para fomentar el trabajo colaborativo, la transparencia en los procesos, la retroalimentación continua y la recopilación de evidencias objetivas sobre la participación.

Los resultados mostraron que, aunque las (calificadores) calificaciones fueron más bajas con el nuevo método (41% frente a 33%), en la distribución de tareas, sobrecargas por parte del docente y hábitos de trabajo poco eficientes.

MeSEP permitió detectar conductos comunes, como la concentración de esfuerzos al final del curso, así como visualizar las contribuciones individuales mediante los (registros de) registros de (GitHub) GitHub.

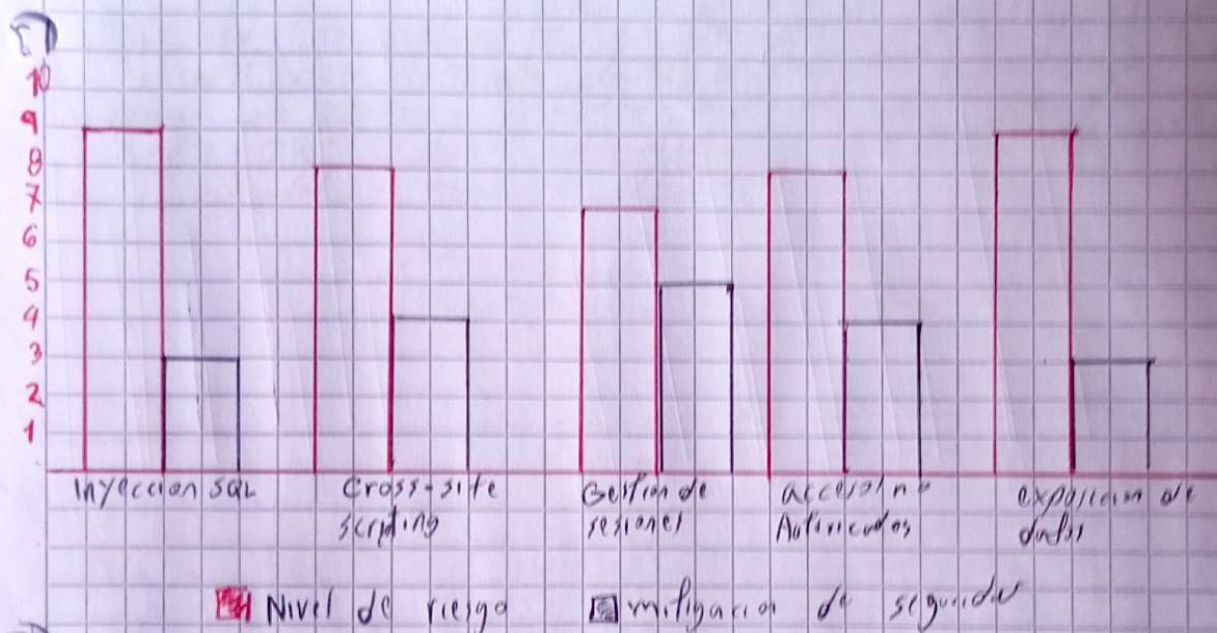


Este estudio evidencia que la tecnología en la educación no es un simple recurso, sino un motor de cambio pedagógico. Su integración fomenta la autonomía y la motivación del estudiante pero también plantea un reto: el docente debe asumir un rol innovador, diseñar y reinventar sus métodos, reflexionando que la verdadera transformación no radica en la herramienta tecnológica, sino en cómo se aprovecha para construir aprendizajes significativos y preparar ciudadanos críticos para un mundo digital.

Artículo 13: Seguridad en aplicaciones web para sistemas en gestión académica

Analiza los riesgos a los que se enfrentan las plataformas académicas de administración (es sensible como calificaciones, asistencia y datos personales). Por su exposición en línea, estos sistemas resultan vulnerables a ataques secuenciales por ser principalmente interfaces SQL, que manipulan consultas para acceder o modificar información y el Cross-site scripting, que introduce código malicioso en el navegador comprometiendo la seguridad de los datos.

Para mitigar esas amenazas, los autores proponen un modelo de detección basado en crawlers y motores de análisis que simulan acciones (maliciosas) (malicio) maliciosas y permiten identificar vulnerabilidades (and) antes que sean explotadas. Así mismo plantea la implementación de un servidor de seguridad interno.



La seguridad en aplicaciones web es esencial cuando se manejan datos académicos sensibles. Este estudio resulta que no basta con construir sistemas funcionales; es necesario garantizar su protección frente a vulnerabilidades. Reflexiona que muchas instituciones educativas descuidan este aspecto, lo que puede comprometer la confianza y la continuidad de los procesos académicos. Por ello, la seguridad debe asumirse como parte integral del diseño y no como un complemento, fomentando una cultura preventiva en el desarrollo tecnológico.

Artículo 14: Aplicaciones web para gestión de grandes eventos

El documento presenta una plataforma web para la venta de entradas y gestión de grandes eventos como conciertos, festivales o congresos, desarrollada con tecnologías PHP, Bootstrap, MySQL, MVC y códigos QR. La propuesta surge ante el creciente uso del comercio electrónico y el auge en España (cerca del 40% en 2016) de la compra de tickets online, especialmente desde dispositivos como ordenadores y smartphones.

La plataforma permite a los usuarios tener un espacio personal donde gestionar sus compras, mantenerse informados sobre eventos favoritos y recibir notificaciones, mientras que los organizadores acceden al panel administrativo para gestionar entradas, noticias y avisos. Además, pueden visualizar estadísticas relevantes mediante gráficos.

El desarrollo se estructura en tres partes principales: (1) plantilla para eventos, con base de datos, diseño web y sistema de compras; (2) espacio de usuario, incluyendo historial, generación de tickets en PDF, código QR; (3) panel administrativo, con gestión de noticias, usuarios, entradas y gráficos de control.

Aplicación

Plantilla para eventos

- Página para eventos
- Registro / login
- Compra de entradas
- Generación PDF/QR
- Responsive

espacio de usuarios

- Historial de compras
- noticias y avisos
- Perfil personal

Espacio de administración

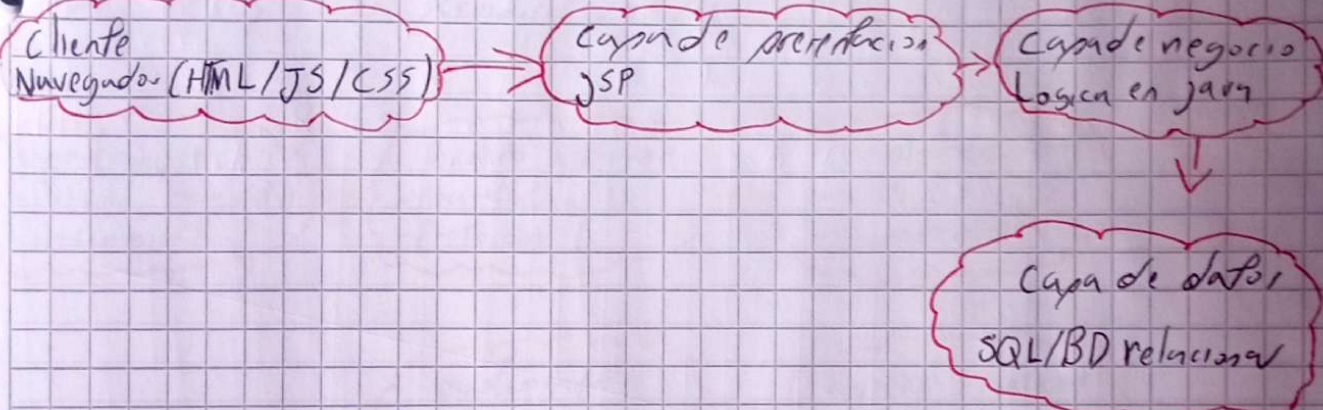
- Gestión de entradas, noticias y avisos
- Gráficos estadísticos
- Gestión de rol
- Administración del usuario

Muestra como una plataforma de venta de entradas online podrá ser funcional, adaptativa y eficiente. El autor alcanza exitosamente los objetivos críticos y secundarios mediante desarrollo ágil y tecnologías abiertas dinámicas. La solución ofrece espacio para usuarios, administradores y eventos, además estadísticas, usuarios otros. Su estructura modular y responsive facilita futura extensión, consolidando una base

Artículo 15: Introducción a las aplicaciones web con Java

Es una guía introductoria a las aplicaciones web con Java dentro del marco de la asignatura Ingeniería del Software. Explica la evolución de la Web, desde los contenidos estáticos en HTML hasta las aplicaciones dinámicas, destacando tecnologías como CGI, JSP. Se describe el modelo cliente-servidor, donde el navegador actúa como cliente y se comunica con un servidor que procesa (sirve) peticiones y devuelve (devuelve) resultados interpretables.

Se profundiza en el papel de JSP (JavaServer Pages) como (frecuente) tecnología clave de J2EE, que permite integrar código Java en páginas HTML para generar contenido dinámico. Se presenta sus elementos básicos y se simplifica su uso en formularios y con JavaBeans, promoviendo la separación entre lógica de negocio y presentación.

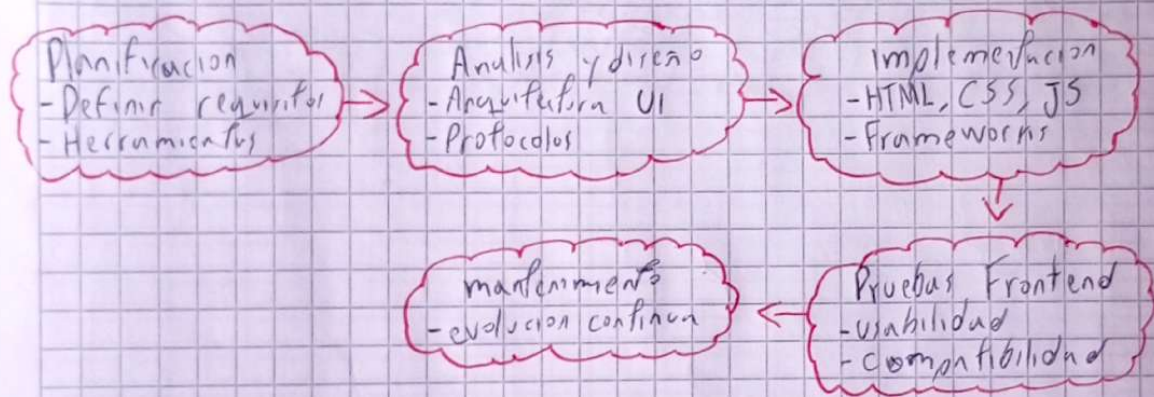


El texto resalta como las aplicaciones web evolucionaron desde modelos cliente-servidor hasta arquitecturas más completas. Reflexiona que esta organización por capas no solo mejora la eficiencia, sino que también fomenta el mantenimiento, la escalabilidad y la reutilización del software. Comprender esta estructura es clave para cualquier desarrollador, ya que permite separar responsabilidades y construir soluciones más claras y robustas, enfrentándose a los retos de proyectos complejos en entornos reales.

Artículo 16: Modelo de proceso para el desarrollo del front-end de aplicaciones web

Presenta una (tecnología) propia metodología para organizar el desarrollo del front-end en aplicaciones web, aplicando el principio de "Divide y vencerás", es decir, separar los procesos del frontend y backend con herramientas, roles y ciclos de vida propios. Dada la dinámica de la ingeniería web con alta interacción, cambios frecuentes y presupuestos limitados se recomienda adoptar modelos iterativos e incrementales frente al modelo de cascada.

Se destaca herramientas clave como frameworks reutilizables, librerías (jQuery, Bootstrap), editores, microservicios, responsive design, task runners y gestores de proyectos como Redmine. Además destaca la importancia de anotaciones para modelar interfaces, eventos y flujos de (II) interacción. Un router con microservicio en Ruby, que desacopla HTML, CSS y JS en servidores distintos.



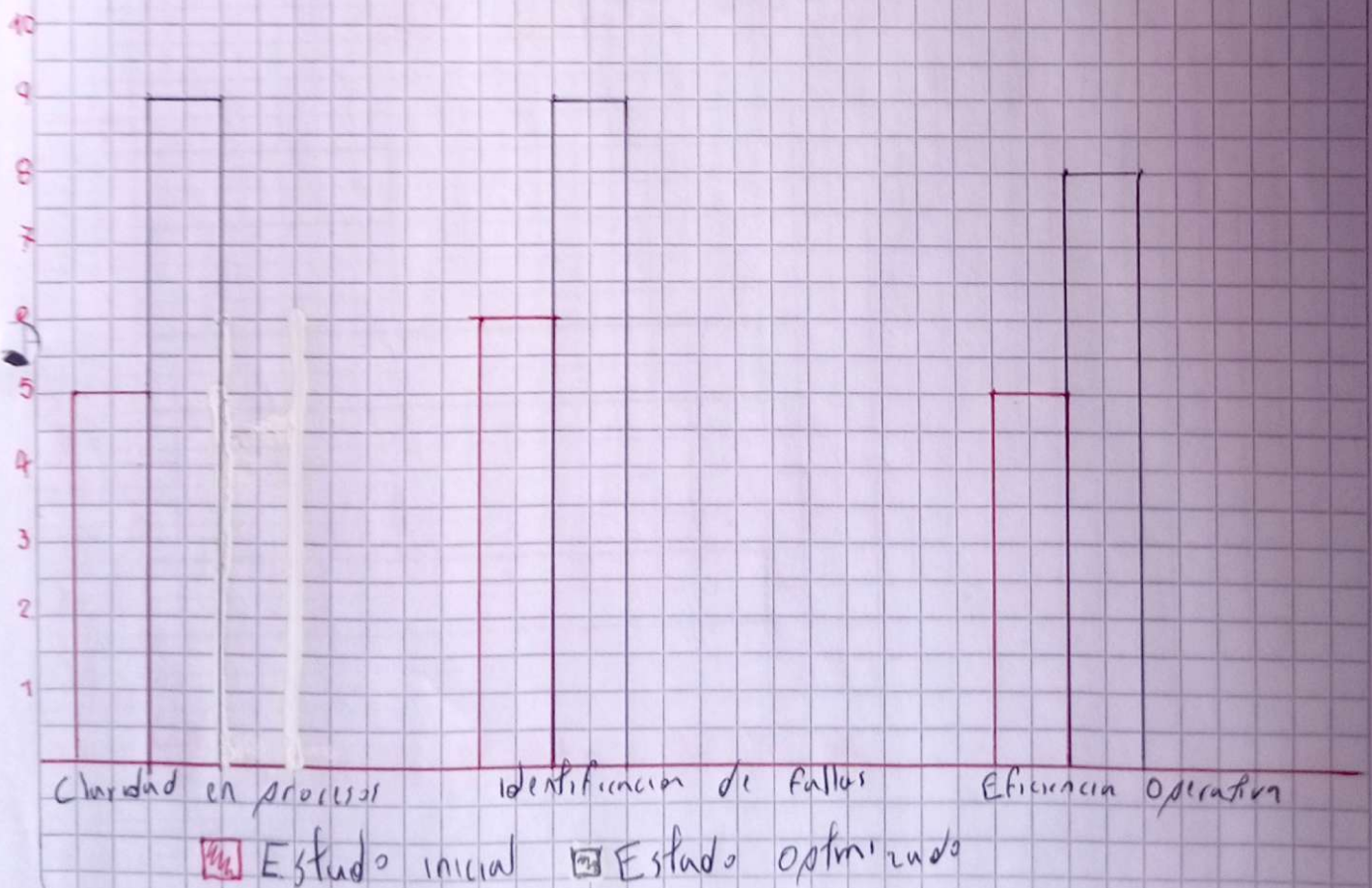
El autor aplica el principio de "divide y vencerás" para proponer un modelo de proceso específico para el desarrollo frontend, vinculándolo al backend y asignándole ciclos de vida y herramientas propias. Este enfoque facilita una implementación más modular, mejor organizada y alineada con estándares. El enfoque refuerza la eficiencia en el diseño de interfaces y promueve mejores prácticas en equipos pequeños, adaptable a múltiples contextos.

Actividad 12: Alternativas para la escalabilidad de aplicaciones en plataformas web de alta concurrencia

El artículo examina la escalabilidad en aplicaciones web Cloud, centrándose en la capacidad de ajustar recursos según la demanda, sin afectar rendimiento ni disponibilidad. (Dispace) Distingue entre escalabilidad vertical, que mejora el hardware de un nodo y horizontal que incrementa el número de nodos, siendo esta última la más empleada en la nube.

Se presentan diversas alternativas frecuentes: balanceadores de carga para distribuir peticiones, replicación de base de datos para la tolerancia a fallos, y almacenamiento distribuido para grandes volúmenes de datos. También se abordan arquitecturas como microservicios, que facilitan un escalado modular y flexible.

El estudio analiza ventajas y limitaciones de proveedores cloud y destaca herramientas de aprovisionamiento automático. Concluye que, junto a la monitorización constante y políticas dinámicas, estas técnicas permiten aplicaciones más robustas, eficientes y adaptables a la demanda.



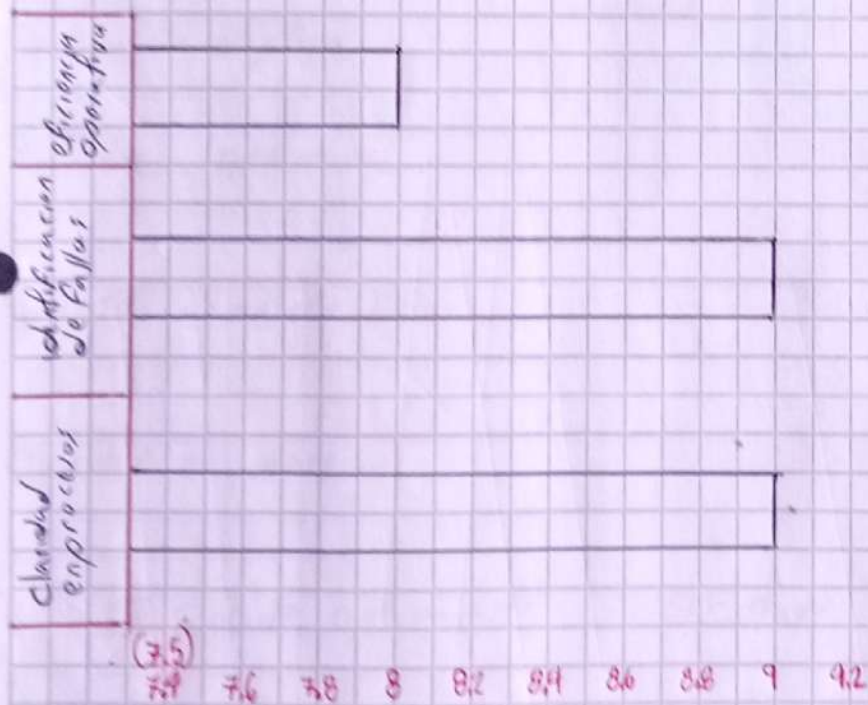
El artículo destaca la importancia de la escalabilidad como atributo esencial en la computación en la nube. Se reflexiona que, ante la necesidad de migrar aplicaciones, es necesario adaptar arquitecturas y servicios para responder al crecimiento dinámico del usuario y datos. Las alternativas presentan desde balances de carga hasta arquitecturas distribuidas, explicando que la escalabilidad no es un lujo, sino un requisito para garantizar eficiencia, disponibilidad y sostenibilidad en entornos digitales (actuales, actuales y futuro).

Artículo 183 Analisis de Frameworks para desarrollo de aplicaciones móviles y web

Análisis comparativo de tres Frameworks de desarrollo multiplataforma: Flutter, React-Native e Ionic, con el propósito de (administrar) identificar cuál resulta más (eficiente) eficiente en términos de desempeño y facilita el uso. Para la evaluación, se diseñó una misma aplicación tipo TO-DO list con funcionalidades como crear, eliminar, ordenar y completar tareas, además de interpretar imágenes, ubicaciones y notificaciones Push. El backend se implementa en Node.js con MongoDB, comunicándose con servicios de Firebase.

Los resultados muestran diferencias importantes. Flutter presenta el mejor rendimiento con bajo consumo de CPU y Ram, además de ofrecer una (curb) curva de aprendizaje corta gracias a su arquitectura basada en widgets y a la integración nativa de librerías. React-native aunque más atractivo para desarrolladores familiarizados con React, muestra un consumo de recursos más alto y dependencias de librerías externas lo que complica la escalabilidad. Por su parte, Ionic destaca por su bajo consumo de CPU, pero registra un uso elevado de memoria y una (arte) estructura de proyectos poco flexible, dificultando el mantenimiento.

Estilo optimizado (0-10)



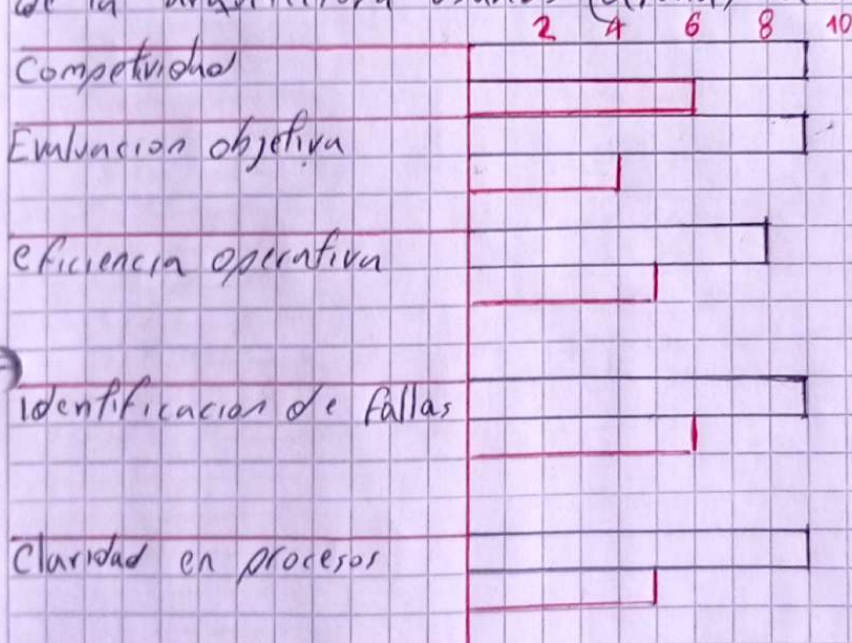
El análisis de frameworks evidencia como estas herramientas son hoy pilares esenciales para acelerar el desarrollo de aplicaciones y garantizar la calidad en la experiencia de usuario. Reflexiono que la elección adecuada no depende solo de la popularidad del framework, sino del contexto del proyecto: escalabilidad, curva de aprendizaje y soporte comunitario. Esta comparación muestra que la innovación en software requiere (vaya) balancear tecnología y necesidades específicas, transformando lo frameworks en aliados estratégicos más que simples recursos.

Artículo 19: Arquitectura basada en microservicios para aplicaciones web

El artículo aborda de forma concisa la arquitectura basada en microservicio aplicada a aplicaciones web, comienza con definiciones (claras) clave sobre este paradigma detallando sus principales características, ventajas y desventajas, así como su estructura arquitectónica, los mecanismos de comunicación entre servicios y su modelo conceptual.

Se destaca como (ventajas) ventajas la escalabilidad, la flexibilidad, el despliegue independiente y la capacidad de aislar fallos. Como desventajas se mencionan la complejidad en la gestión, mayor sobrecarga en la gestión, mayor sobrecarga operativa y retraso en la coordinación entre servicios. El artículo también diferencia este enfoque de la arquitectura monolítica tradicional, subrayando como los microservicios permiten (organizar) organizar funciones individuales como unidades independientes que interactúan entre sí.

Además el documento hace referencia a aspectos de comunicación síncrona y asíncrona, necesario para garantizar la concurrencia y la eficiencia en las aplicaciones distribuidas. También el modelo de la arquitectura usando (etiquetas) anotaciones apropiadas.



■ Estado optimizado

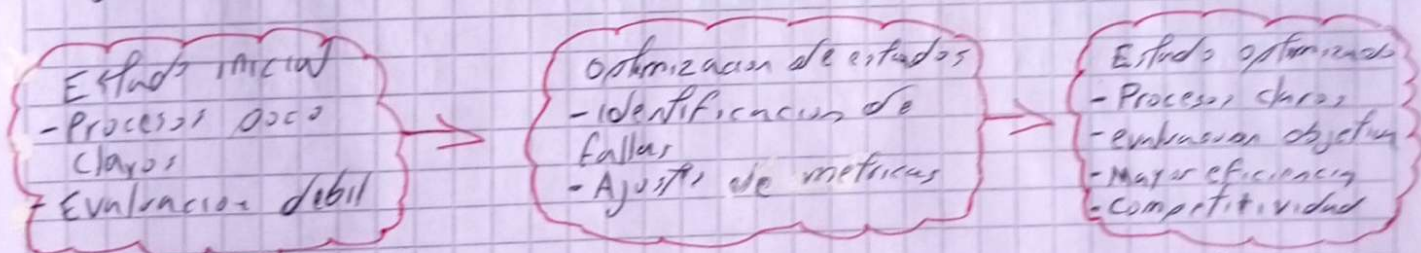
■ Estado inicial

Este artículo introduce de manera clara la arquitectura de microservicios para aplicaciones web, destacando su modularidad, escalabilidad y flexibilidad. Reflexiona que este enfoque representa un cambio fundamental frente a los sistemas monolíticos al permitir (desarrollar) desarrollar, escalar y desplegar de forma independiente. Sin embargo, también introduce retos en la (comunidad) comunicación entre servicios y en la (comunidad) complejidad operativa. En conjunto, plantea una opción potente y moderna valiosa para entornos dinámicos y propósitos de gran escala.

Artículo 20: Optimización de estados en la mejora de procesos de software

El artículo propone un enfoque de optimización de estados para mejorar los procesos de software en organizaciones que busquen mayor eficiencia y calidad. Se parte del reconocimiento de que muchos (procesos) modelos de mejora, aunque efectivos, suelen ser complejos, costosos y difícil de implementación en pequeñas y medianas empresas. Ante ello, los autores plantean una alternativa más (práctica) práctica, enfocada en identificar y optimizar los estados clave de los procesos, entendidos como las condiciones intermedias que marcan la evolución hacia un producto o servicio final.

La metodología consiste en definir estos estados, analizarlos con base en métricas y seleccionar aquellos que aporten mayor valor al negocio. De esta manera, la optimización no se centra en rediseñar el proceso completo, sino ajustar fases críticas donde se generan ineficiencias.



El texto resalta que la mejora de procesos de software debe enfocarse en la optimización de estados dentro de los modelos de calidad. Reflexiona que este enfoque permite identificar con mayor precisión sus fortalezas y debilidades, logrando planes de mejora más efectivos. Este enfoque demuestra que la optimización no es solo un técnica, sino estrategia para la madurez y competitividad en el desarrollo de software.